Project 4

Kagan Akcelik (ka33824)
Cody Chang (cwc2859)

Section 1. Overview

In this assignment, we implemented the core game logic for a version of Tetris, along with an AI "brain" that plays the game on its own. The provided JTetris class handles the GUI, timing, and scoring, while our code handles everything about the game's state. Our implementation consists of four main pieces:

- TetrisPiece: an immutable representation of one tetromino in one rotation, with constant-time access to its rotations, body, and skirt.
- TetrisBoard: the board state. It stores placed blocks, moves and rotates the falling piece (including Super Rotation System wall kicks), places pieces, clears rows, and answers questions about the board in constant time.
- TetrisBrain: an AI that enumerates every place the current piece can land, scores each resulting board, and moves toward the best one.
- JBrainTetris: a subclass of JTetris that lets the brain play instead of the keyboard.

The main goal of this assignment was decomposing a large problem (a whole Tetris game) into smaller pieces that could be built and tested independently. It also emphasized design decisions, since the internals of the Board were left entirely up to us.

We decomposed the game into layers that each depend only on the one below it through an interface: TetrisPiece knows nothing about the board, TetrisBoard only interacts with pieces through the Piece interface, and the brain only interacts with the board through the Board

interface (mainly `testMove` and the accessors). This abstraction meant each layer could be tested on its own before the next was built on top of it.

Section 2. Scope, Quality, and Limitations

Our implementation supports all seven standard tetrominoes and all actions defined by the Board interface: LEFT, RIGHT, DOWN, DROP, CLOCKWISE, COUNTERCLOCKWISE, and NOTHING. Rotations follow the Super Rotation System, including wall kicks: the J, L, S, T, and Z pieces use the normal kick table, the I piece uses its own, and the O piece rotates in place since its shape never changes. Row clearing follows the spec's rule that blocks only shift down by the number of cleared rows below them, so blocks can appear to float above holes.

Every accessor the spec requires to be constant time is constant time: getLastAction, getLastResult, getRowsCleared, getWidth, getHeight, getMaxHeight, getRowWidth, getColumnHeight, and getGrid. Every Piece operation except construction is also constant time, including rotation.

Known correctness issues and limitations:

- nextPiece does not check whether the spawn position is blocked. If a piece spawns overlapping placed blocks, placing it would overwrite them. In practice, JTetris ends the game before this happens.
- After a piece is placed, getCurrentPiecePosition still returns the last position of the placed piece instead of null.
- The brain only considers straight drops. It never slides a piece under an overhang or performs T-spins, and it does not look ahead at the next piece.
- The brain's weights were tuned by hand, not optimized.

Section 3. Algorithm & Implementation

TetrisPiece

Each TetrisPiece represents one piece in one specific rotation, so each tetromino has four TetrisPiece objects. These four are linked in a circular, doubly linked ring: each piece stores a pointer to its clockwise and counterclockwise neighbors. Because the ring is built once, in the constructor, clockwisePiece() and counterclockwisePiece() just return a pointer, which makes rotation constant time.

Constructor

The public constructor builds rotation 0 from the piece type's spawn body, then walks clockwise around the ring four times. The first time clockwisePiece() is called on a rotation, it creates the next rotation by rotating every block 90° clockwise inside the square bounding box with the formula (x, y) → (y, width − 1 − x). When rotation 3 is reached, its clockwise neighbor is linked back to rotation 0, closing the ring. Each newly created rotation also sets its counterclockwise pointer, so the ring is linked in both directions.

Skirt
findSkirt() builds an array as wide as the bounding box, fills it with Integer.MAX_VALUE (meaning an empty column), and then, for every block, keeps the lowest y value in that block's column.

equals
Two pieces are equal when they have the same type and rotation index. That fully determines the body, so the bodies don't need to be compared.

For TetrisBoard

State
The board is stored as a 2D array of PieceType indexed board[y][x], where null means an empty cell. Only placed blocks are stored in the grid. The falling piece lives separately in currentPiece and currentPosition, where the position is the bottom-left corner of the piece's bounding box and can be negative. Alongside the grid, we store rowWidths (filled cells per row), columnHeights (the highest block's y + 1 per column), maxHeight, rowsCleared, lastAction, and lastResult.

move(Action act)
This method records the action, resets rowsCleared to 0 (it only counts rows cleared by the current move), and returns NO_PIECE if there is no falling piece. Otherwise, a switch statement handles each action:

- LEFT and RIGHT try to shift the piece by one column. If it doesn't fit, they return OUT_BOUNDS and the piece does not move. Hitting a wall sideways never places a piece.
- DOWN moves the piece down one row if it fits. Otherwise there is something directly below the piece, so it is placed and the method returns PLACE.
- DROP moves the piece down one row at a time until it is blocked, then places it.

- CLOCKWISE and COUNTERCLOCKWISE rotate the piece using wall kicks (see setRotation).
- NOTHING returns SUCCESS.

setPosition(Piece p, Point position)
This is the collision check that every movement goes through. It adds the position to each block of the piece's body and checks that every resulting cell is on the board and empty. If they all are, it updates the current piece and position and returns true. Collision is checked against the piece's actual blocks, not its bounding box, so a vertical I piece against the left wall can have its bounding box hang off the board.

setRotation(Piece p, Point position, Point[] wallKicks)
This method tries each wall-kick offset from the provided SRS tables in order. For each offset, it checks whether the rotated piece fits at the current position plus that offset. The first offset that fits is applied, and the method returns true. If none fit, the rotation fails and the piece is unchanged. The tables are selected by the piece's current rotation index, and the I piece uses its own tables.

placePiece()
This method writes the current piece's blocks into the grid and updates the bookkeeping for each block: the row's width increases by one, the column's height becomes the maximum of its current height and the block's y + 1, and maxHeight is updated. We use Math.max for column height instead of incrementing, because a column's height depends on where its top block is, not how many blocks it contains. A column with one block floating above a hole can still be several rows tall. After writing the blocks, it calls clearRows() and sets the current piece to null.

clearRows()
This method compacts the board in a single pass, using two indices. read scans every row from bottom to top, and write marks where the next kept row goes. A full row is counted and skipped. A non-full row is moved down to the write slot, and write advances. Because the grid is indexed by row first, moving a row is a single reference assignment rather than copying each cell. Any slots left at the top become fresh empty rows. If any rows were cleared, column heights are recomputed by starting at each column's old height and walking down to the first block. This is necessary because a column's height can drop by more than the number of rows cleared: if its top block was in a cleared row and there was a hole beneath it, the new top can be much lower.

dropHeight(Piece piece, int x)
This method returns the y position the piece's bounding box would land at if dropped straight down at column x. For each column of the piece, the piece must sit high enough that its lowest block in that column (the skirt) rests on top of the board's column, so the answer is the maximum of columnHeight − skirt across the piece's columns. Columns where the skirt is Integer.MAX_VALUE are skipped, since the piece has no block there. The result starts at Integer.MIN_VALUE rather than 0 because it can be negative: a T piece's bounding box has an empty bottom row, so on an empty board its box lands at y = −1.

testMove(Action act)
This method creates a full copy of the board with a private copy constructor, applies the move to the copy, and returns it. The original board is not modified.

equals(Object other)
Two boards are equal when they have the same dimensions, every grid cell matches, and they have the same falling piece at the same position. Position is only compared when there is a falling piece.

Design Decisions
To make every accessor constant time, we store each answer that would otherwise require looping over the board in its own field: rowWidths, columnHeights, maxHeight, rowsCleared, lastAction, and lastResult. The grid and these counts only change when a piece is placed, so all the expensive work happens in one place, and every other operation stays cheap. Rows are cleared immediately on placement, since that's the only time a row can become full. Clearing is done in a single bottom-to-top pass that moves whole row references down instead of copying cells.

For testMove, the copy shares the original's piece objects, since pieces are immutable, but gets its own copy of the grid, every row, the count arrays, and the position. Sharing any of those would mean a move on the copy also changes the original.

dropHeight uses the piece's skirt (the lowest block in each column) together with the board's column heights. The piece lands at the highest point where every one of its columns rests on the board.

TetrisBrain

The brain enumerates every place the current piece can land, scores each resulting board, and moves toward the best one. For each of the four rotations, it uses testMove to simulate dropping the piece at every reachable column (sliding left and right until it hits a wall). Each board is scored as: score = 100 − gaps − 3 × maxHeight − bumpiness

- Gaps are empty cells beneath the top of a column. Holes are hard to clear, so they're penalized.
- Max height is weighted three times as heavily as the others, because a tall stack is what ends the game.
- Bumpiness is the total height difference between neighboring columns. Flat surfaces fit pieces cleanly, while deep wells need an I piece.

Line clears are rewarded indirectly, since clearing rows lowers the stack and often removes gaps.

nextMove
returns the first action toward the highest-scoring board: CLOCKWISE if it needs a different rotation, otherwise LEFT, RIGHT, or DROP. JBrainTetris calls the brain repeatedly after each piece spawns, so the brain re-plans after every action and takes one step at a time. The best score starts at Integer.MIN_VALUE, since scores go negative as the board fills.

JBrainTetris
JBrainTetris extends JTetris so that the brain plays the game instead of the keyboard. It creates a TetrisBrain and overrides addNewPiece(): after JTetris spawns a new piece, JBrainTetris repeatedly asks the brain for its next move with nextMove(board) and applies it through JTetris's tick() method, until the piece has been placed. Because it reuses tick() and only overrides addNewPiece(), everything else (the GUI, piece generation, scoring, and game-over detection) is inherited unchanged from JTetris, so the brain plays under exactly the same rules as a human player.

Comparison to LameBrain
Our brain is built on the same structure as the provided LameBrain: it enumerates landing spots with testMove, scores each resulting board, and returns the first move toward the best one. However, it improves on LameBrain in two major ways. First, LameBrain never rotates pieces, so it only considers each piece in its spawn orientation. Our brain considers all four rotations, which roughly quadruples the

number of options it can choose from. Second, LameBrain scores boards only by max height (100 − 5 × maxHeight), so it can't tell the difference between two boards of the same height even if one is full of holes. Our brain also penalizes gaps and bumpiness, so it prefers boards that are flat and solid, not just short. We also changed the starting best score from 0 to Integer.MIN_VALUE. With our scoring formula, scores can go negative partway through a game, and LameBrain's starting value of 0 would then cause the brain to always choose its first option.

Section 4. Testing

Testing was done with JUnit. The real-time nature of the game makes testing through the GUI unreliable: pieces fall on a timer, spawn randomly, and the board's internal state isn't visible. To get around this, our tests create boards directly and use nextPiece to put a specific piece at a specific position. This lets us set up exact board states, like a nearly full row or a piece against a wall, and check the results by hand.

Our general tests (TetrisTest) cover:

- Piece properties: type, bounding box size, and that the skirt and body exist.
- Rotation: rotating clockwise gives rotation index 1, and four clockwise rotations return to the original rotation.
- Board initialization: correct width and height, a max height of 0, and no current piece.
- Moving left and right updates the piece's position correctly.
- DROP returns PLACE, removes the current piece, and raises the max height.
- The brain returns a valid action for a given board.

Our edge-case tests (BlackBoxTetrisBoardTest) target the situations most likely to hide bugs, using only the public Board interface:

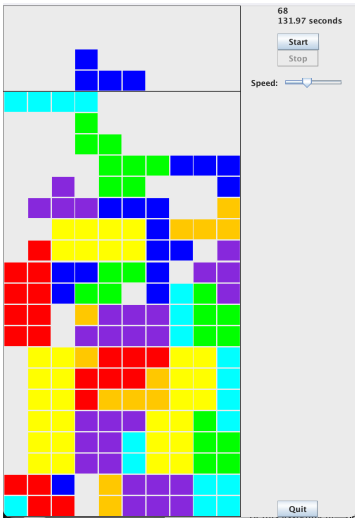
- A vertical I piece's bounding box hanging off the left wall, which checks that collision uses the actual blocks rather than the bounding box.
- Hitting a wall sideways returning OUT_BOUNDS without placing the piece.
- DOWN placing a piece on top of another block instead of the floor.
- Column heights over holes.
- A column's height dropping all the way to 0 after a clear.
- A four-row clear shifting the rows above down by exactly four.

- A wall kick off the left wall.
- The O piece rotating in place.
- dropHeight returning negative values and skipping empty skirt columns.
- testMove leaving the original board unchanged.

The biggest lesson we learned was that the GUI hides the real cause of bugs. At one point, every block above row 10 disappeared from the screen. That looked like a drawing problem, but it was actually a typo in getGrid's bounds check. Setting up exact board states with nextPiece made bugs like this much easier to pin down than playing the game and trying to recreate a situation by hand. We also found that the most valuable tests target edge cases (walls, holes, and multi-row clears) rather than normal play, since the normal cases worked almost immediately, while the bugs lived at the boundaries.

Section 5. Interesting Results

Watching the brain play was interesting, because its priorities were visible in how it stacked pieces. It strongly prefers keeping the surface flat, and it avoids leaving holes even when that means placing a piece somewhere a human might not. Before we fixed the bug where the best score started at 0, the brain played noticeably worse. Once the stack got moderately tall, every score became negative, so the brain fell back to dropping every piece straight down wherever it spawned. Fixing that one line made a visible difference in how long it survived.



This was one of the better runs we had while playing. As you can probably tell, we are not too great at playing tetris.

Section 6. Assumptions

We assumed that the provided JTetris, Board, Piece, and PieceType code behaves as documented, including the SRS wall-kick tables in the Piece interface being indexed by the piece's starting rotation. We also assumed that JTetris handles game-over detection using getMaxHeight(), so the board does not need to detect a game over itself.

Section 7. Problems Encountered

We had issues with writing the falling piece into the grid. Our first version of the board erased and repainted the falling piece in the grid on every move. This made the piece collide with its own blocks when moving down, risked erasing placed blocks, and made the row and column counts include the falling piece. We fixed it by keeping the falling piece completely separate from the grid and only writing it in when it is placed.

We also had issues with blocks disappearing above row 10. A typo in getGrid's bounds check compared y against the board's width instead of x. Since the board is 10 columns wide, every row from 10 up was reported as empty, so those blocks never appeared on screen.

Another issue was git merge conflicts. We committed compiled .class files to our repository, which caused binary merge conflicts. More than once, resolving a conflict by resetting to the remote version overwrote one partner's local work. We learned to keep build output out of version control, back up files before resolving conflicts, and tell each other before editing a shared file.

Section 8. Partner Log

We used VS Code Live Share to work in the same coding session from our own computers, rather than assigning a fixed driver. Our estimated time breakdown was:

October 2: Together: 1 hour; Cody individually: 15 minutes; Kagan individually: 0 minutes.
October 3: Together: 30 minutes; Cody individually: 30 minutes; Kagan individually: 0 minutes.

October 5: Together: 2 hours; Cody individually: 30 minutes; Kagan individually: 30 minutes.
October 7: Together: 2 hours 30 minutes; Cody individually: 45 minutes; Kagan individually: 30 minutes.
October 9: Together: 45 minutes; Cody individually: 0 minutes; Kagan individually: 30 minutes.
October 11 25: Together: 2 hours; Cody individually: 15 minutes; Kagan individually: 45 minutes.
October 13 25: Together: 1 hour; Cody individually: 15 minutes; Kagan individually: 30 minutes.
October 15 25: Together: 1 hour; Cody individually: 15 minutes; Kagan individually: 30 minutes.
October 16 25: Together: 3 hours; Cody individually: 15 minutes; Kagan individually: 30 minutes.

Working together was helpful because we could explain our reasoning and discuss problems while looking at the same code. Compared with working alone, having another person question an assumption made it easier to notice cases we had overlooked. However, agreeing on an implementation did not necessarily mean it was correct, so testing was still necessary.

Section 9. Social Impact

Tetris is one of the most widely played games in the world, but its standard design assumes the player can see the screen clearly, tell colors apart, and press keys quickly. That leaves a lot of people out. Color blindness alone affects roughly 1 in 12 men and 1 in 200 women, and Tetris pieces are distinguished mainly by color. Players with motor impairments may not be able to use a keyboard at all, especially under time pressure, and blind or low-vision players can't play a purely visual game.

Accessibility matters in technology because a design choice that seems neutral can completely exclude people, and the fixes often aren't difficult if they're considered from the start. For Tetris, there are several improvements we could make:

- Color: give each piece a distinct pattern or symbol in addition to its color, and offer colorblind-friendly palettes.

- Sound: play distinct audio cues for moving, rotating, landing, and clearing lines, and use stereo panning to indicate where the piece is horizontally. This would make the game at least partly playable without seeing the screen.
- Input and UI: allow remappable controls, support alternative input devices like Microsoft's Xbox Adaptive Controller, and let players slow the drop speed or switch to a turn-based mode where pieces only fall when the player acts.

Interestingly, the brain we built for this project points toward another option: an assisted mode, where the AI suggests the best placement or handles fine positioning while the player chooses where pieces go. Efforts like the Game Accessibility Guidelines and the W3C's WCAG standards show that the field is starting to treat accessibility as a requirement rather than an afterthought. As developers, we should design for every type of user from the start, not add accessibility as a patch later.

Sources:

- https://tetris.wiki/Tetris.wiki
- https://howtotetris.com/how-tetris-really-works/
- https://ablegamers.org/how-the-gaming-industry-is-adapting/